

SERGIO ALEJANDRO ARIAS MAYTA

LINK DEL REPOSITORIO:

https://github.com/SaiPlusSai/Sergio_Arias_2daEvaluacionBackend.git

1. Análisis del Proyecto Integrador (15 pts)

Identificación clara del endpoint (5 pts)

Endpoint consumido:

- GET /api/historial-busqueda/google-like?frase=alimentacion saludable

Este endpoint forma parte del proyecto integrador MIGA y permite realizar una búsqueda tipo Google en los campos descripcion y relevancia.

Justificación de la elección del endpoint (5 pts)

- Es uno de los endpoints más importantes del sistema, ya que implementa un sistema de búsqueda inteligente que analiza frases completas.
- Es ideal para ser reutilizado mediante un microservicio, ya que puede aportar valor a otros sistemas que requieran consultas por frases.
- Sirve como ejemplo real de procesamiento y transformación de datos, lo que lo vuelve adecuado para aplicar tecnologías como **SOAP**.

Descripción técnica del endpoint

Característica	Detalle
Método	GET
URL	/api/historial-busqueda/google-like
Parámetros	frase (string): frase completa para buscar
Campos utilizados	descripcion, relevancia (dentro de los documentos vigentes)
Formato de respuesta	XML: lista de documentos que hacen match con al menos una palabra clave

2. Diseño del Microservicio (15 pts)

Objetivo

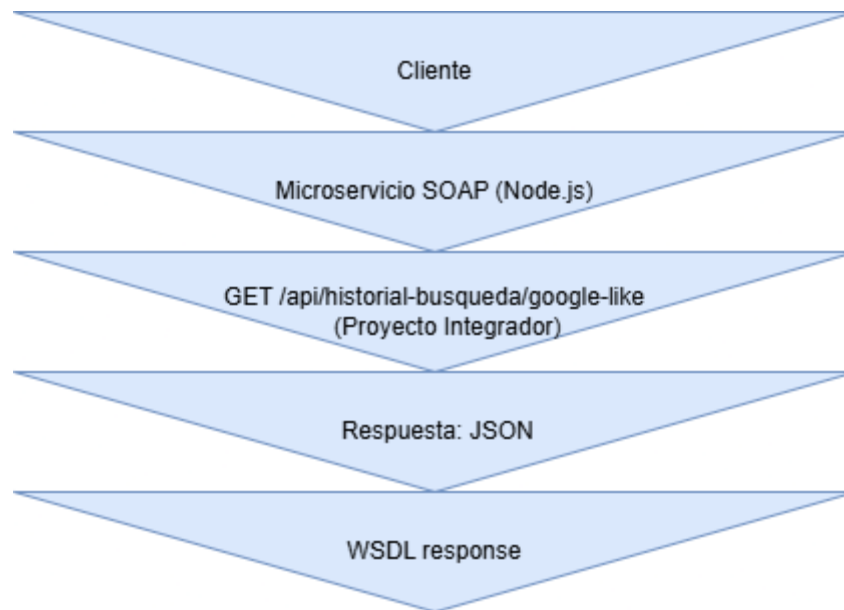
Construir un microservicio SOAP que permita consultar frases a través del endpoint google-like, procesando los resultados y devolviéndolos como una cadena JSON en el formato de respuesta SOAP.

Elección de la tecnología de comunicación (5 pts)

Se eligió **SOAP con WSDL** porque:

- Era un requisito del parcial demostrar una tecnología no REST.
- SOAP es ampliamente usado en sistemas interoperables de gobierno y salud.
- Permite definir claramente contratos de servicio mediante archivos .wsdl.

Diagrama del flujo de integración



3. Implementación Técnica

Desarrollo del microservicio funcional

- Se creó un archivo `soapServer.js` que corre en el puerto 8001.
- Se expone la operación `buscarFraseGoogle`.

Consumo correcto del endpoint externo

- Se utiliza `axios` para realizar la petición a `http://localhost:3000/api/historial-busqueda/google-like`.

Procesamiento y transformación de los datos

- Los datos se convierten en `JSON.stringify()` antes de retornarlos como respuesta en el SOAP.

Exposición de endpoint propio/documentado

- Endpoint SOAP expuesto en `http://localhost:8001/wsdl`.
- Archivo `wsdl/busqueda.wsdl` contiene la definición estructurada.

Estructura del proyecto

/MicroservicioSOAP

```
|
|
|— soapServer.js
|— wsdl/
|   └─ busqueda.wsdl
|— package.json
|— README.md
└─ .gitignore
```

Codigo base

1.busqueda.wsdl

```
<!-- wsdl/busqueda.wsdl -->
<definitions name="BusquedaServicio"
  targetNamespace="http://www.ejemplo.org/busqueda"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://www.ejemplo.org/busqueda"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <message name="buscarFraseGoogleRequest">
    <part name="frase" type="xsd:string"/>
  </message>
  <message name="buscarFraseGoogleResponse">
    <part name="resultado" type="xsd:string"/>
  </message>

  <portType name="BusquedaPort">
    <operation name="buscarFraseGoogle">
      <input message="tns:buscarFraseGoogleRequest"/>
      <output message="tns:buscarFraseGoogleResponse"/>
    </operation>
  </portType>
```

```

<binding name="BusquedaBinding" type="tns:BusquedaPort">
  <soap:binding style="rpc"
    transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="buscarFraseGoogle">
    <soap:operation soapAction="buscarFraseGoogle"/>
    <input>
      <soap:body namespace="http://www.ejemplo.org/busqueda" use="literal"/>
    </input>
    <output>
      <soap:body namespace="http://www.ejemplo.org/busqueda" use="literal"/>
    </output>
  </operation>
</binding>

<service name="BusquedaServicio">
  <port name="BusquedaPort" binding="tns:BusquedaBinding">
    <soap:address location="http://localhost:8001/wsdl"/>
  </port>
</service>
</definitions>

```

2.soapServer.js

```

import express from 'express';
import soap from 'soap';
import axios from 'axios';
import http from 'http';
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import js2xmlparser from 'js2xmlparser'; // 🆕 NUEVO

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const app = express();

const servicio = {
  BusquedaServicio: {
    BusquedaPort: {
      async buscarFraseGoogle(args, callback) {
        const { frase } = args;

        try {

```

```

        const response = await axios.get('http://localhost:3000/api/historial-busqueda/google-like', {
            params: { frase }
        });

        // 🔄 Convertir array de resultados a XML bonito
        const xml = js2xmlparser.parse("documentos", { documento: response.data
    });

    callback({ resultado: xml });
    } catch (err) {
        callback({ resultado: js2xmlparser.parse("error", { mensaje: "Error en el microservicio" }) });
    }
    }
    }
    }
    }
    });

const server = http.createServer(app);
const wsdlPath = path.join(__dirname, 'wsdl', 'busqueda.wsdl');
const wsdlXml = fs.readFileSync(wsdlPath, 'utf8');

soap.listen(server, '/wsdl', servicio, wsdlXml);

server.listen(8001, () => {
    console.log('🚀 Microservicio SOAP corriendo en http://localhost:8001/wsdl');
});

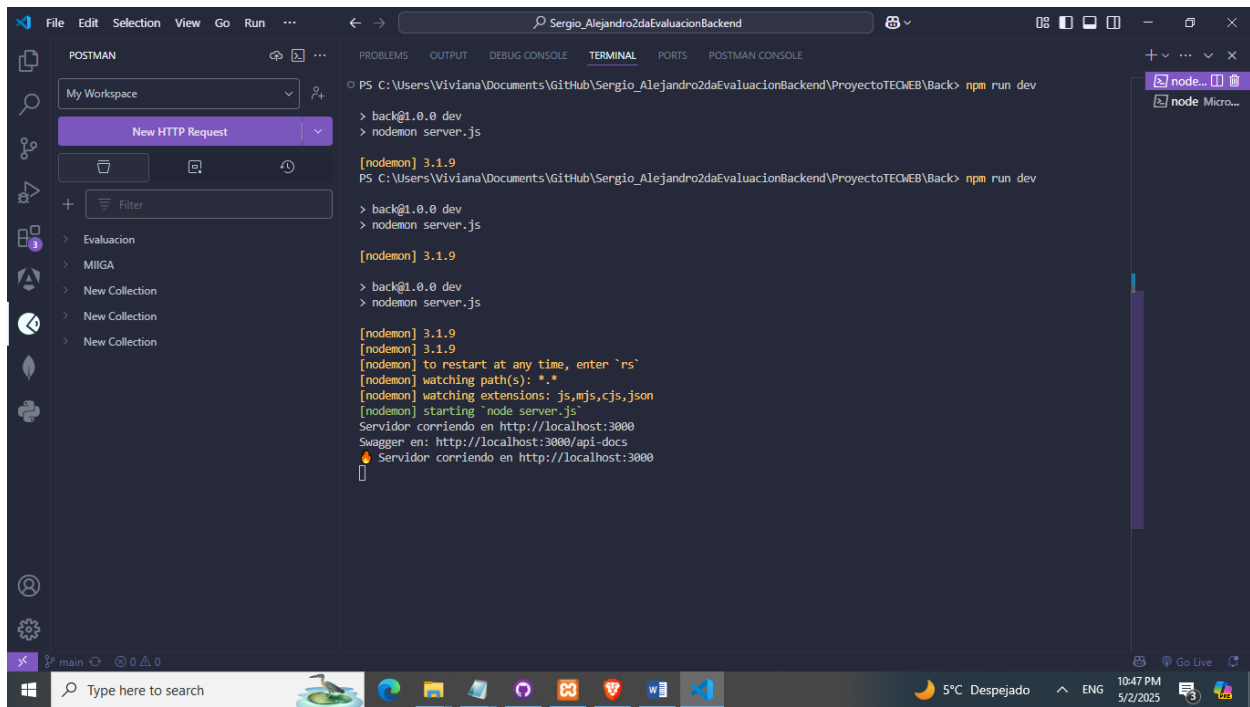
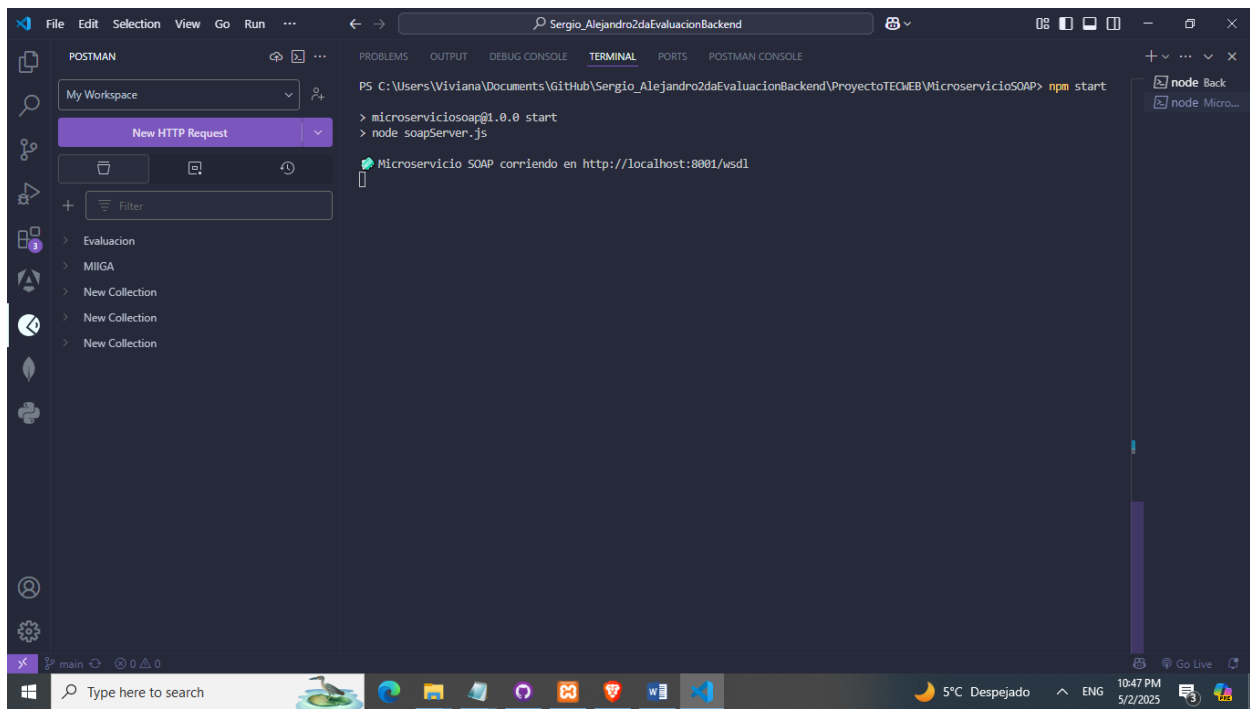
```

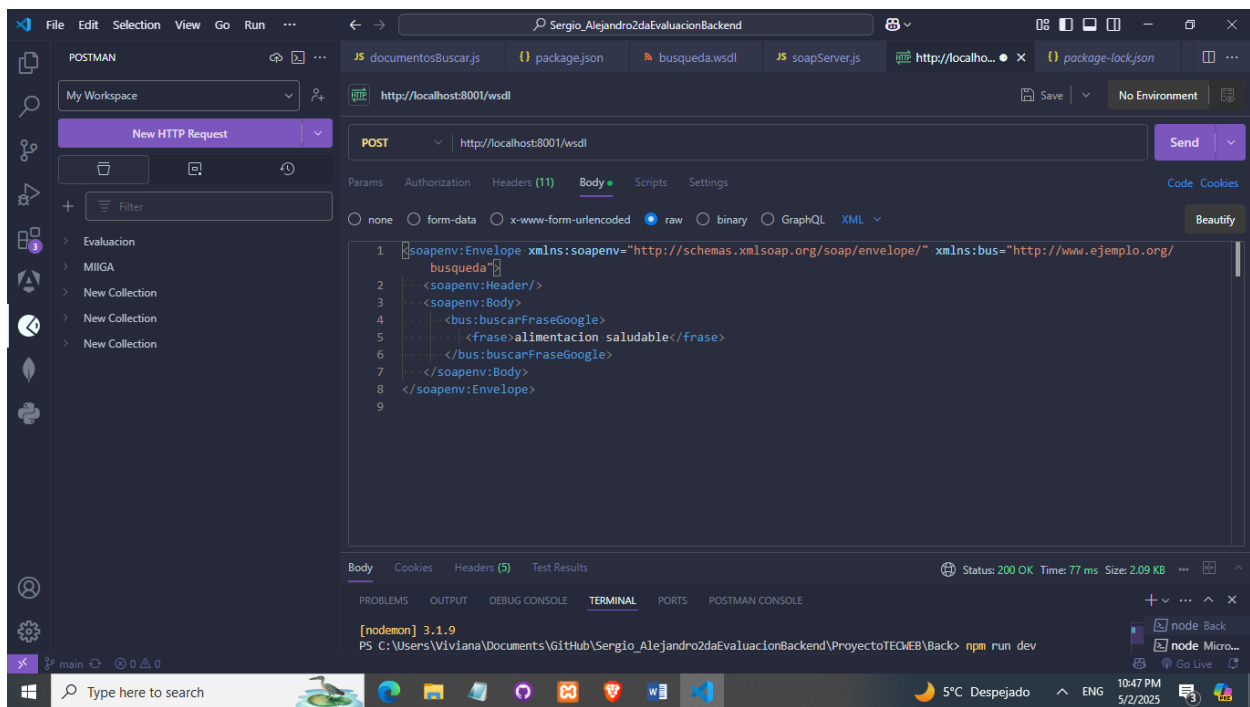
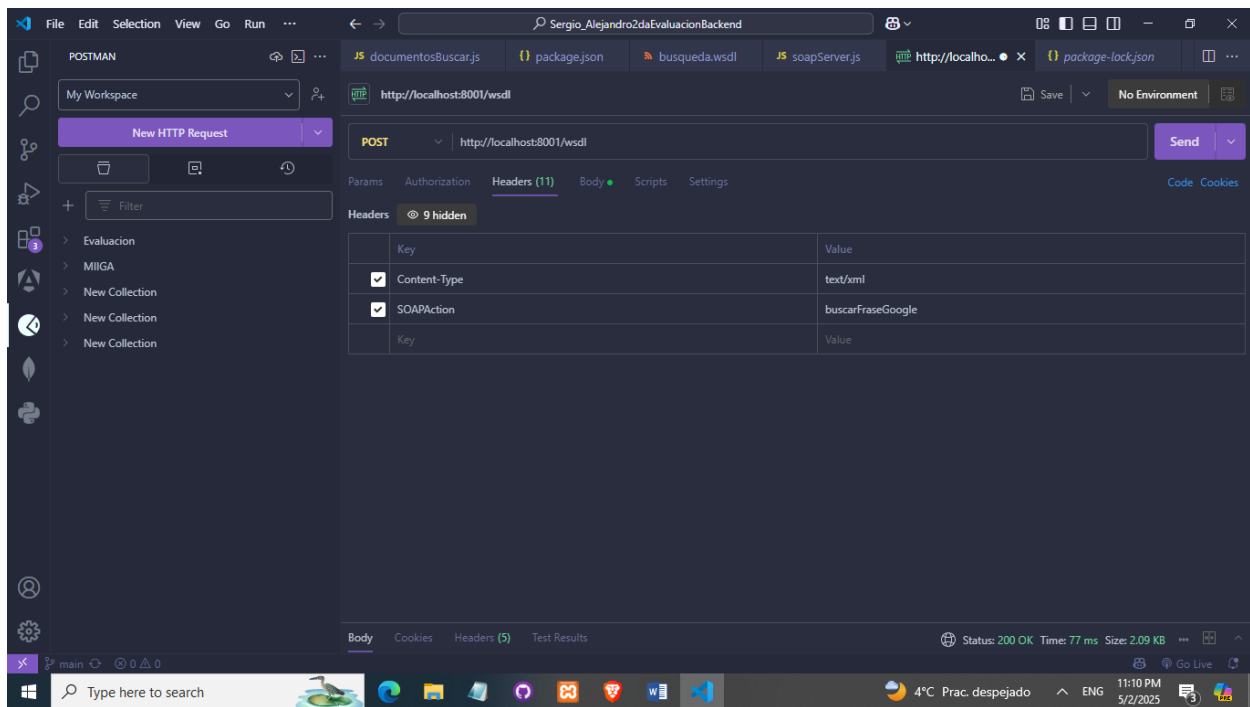
Librerías necesarias: npm install soap express axios

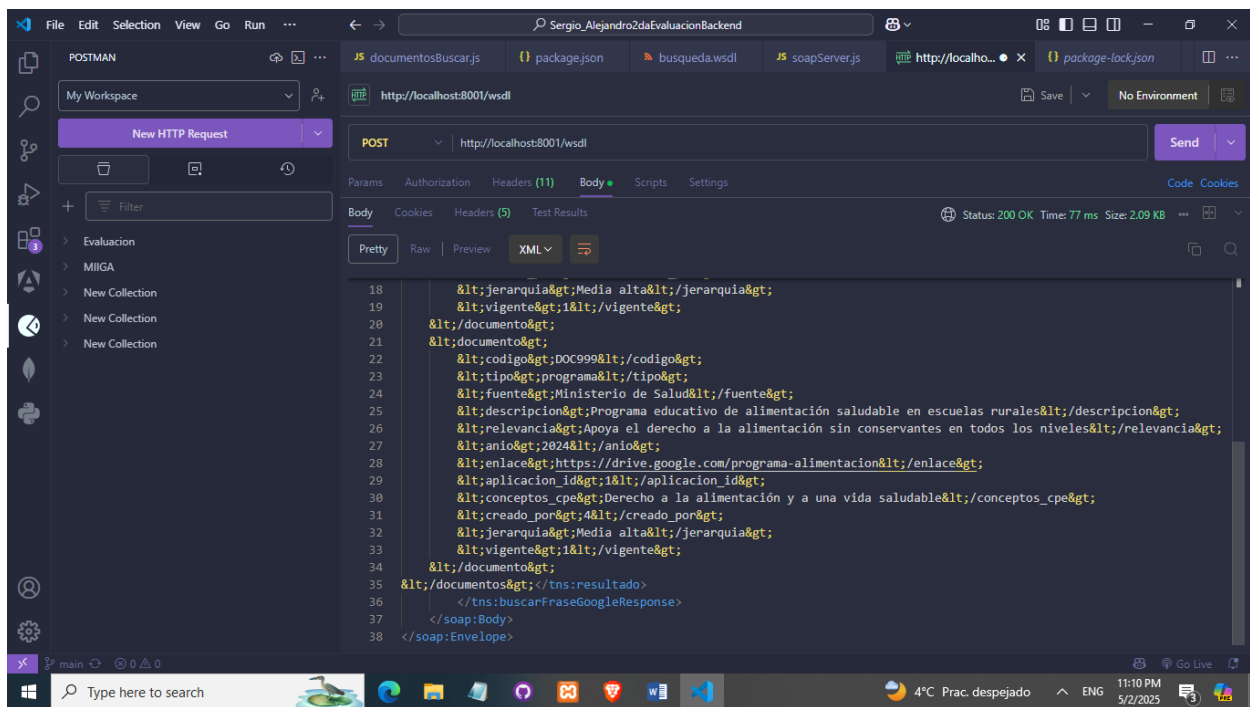
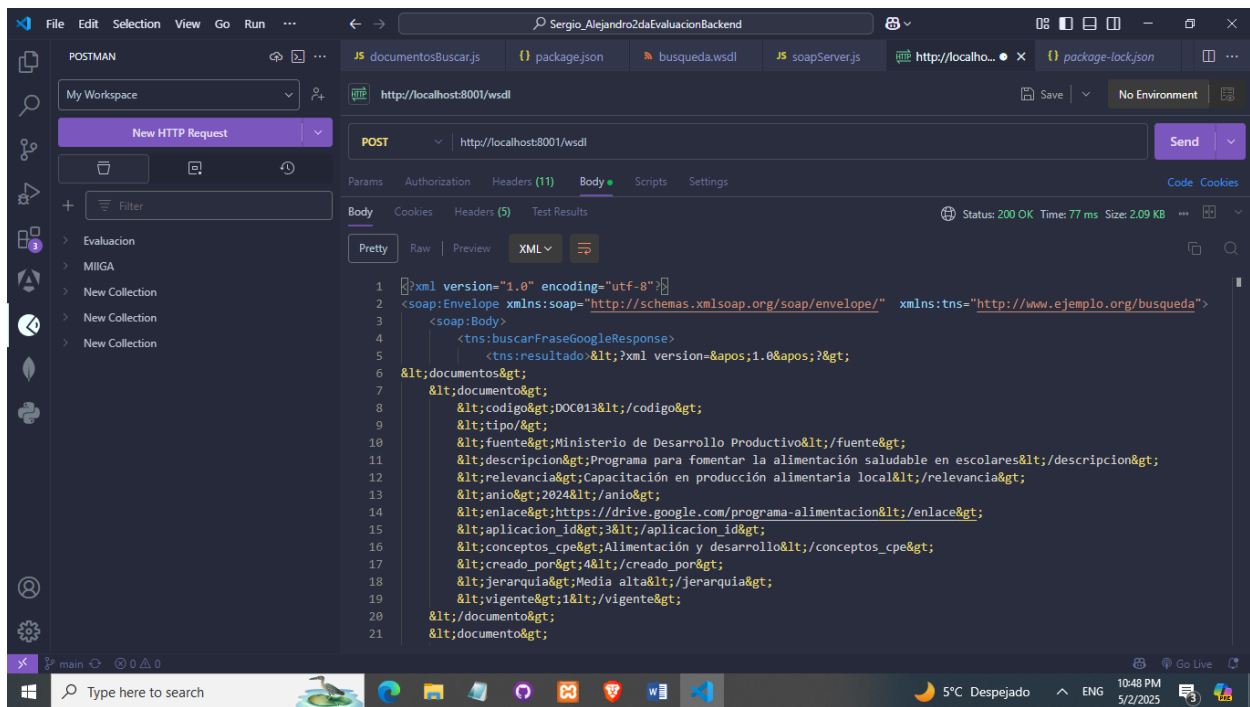
4. Pruebas y Documentación

Evidencias funcionales

- Peticiones realizadas desde **SOAP Client en Postman:**
 - Método: POST
 - Header: Content-Type: text/xml
 - Body: XML de solicitud con `<frase>alimentacion escolar</frase>`
- Respuesta estructurada como XML y muestra los documentos coincidentes.







✓ Pruebas manuales explicadas (5 pts)

- Prueba con frases completas y verificación visual del retorno de documentos.
- Prueba de error sin parámetro `frase`, retornando status 400.

